
ARVR Insurance

Project Report

Using AR/VR Technology to Sell Life Insurance

Framework	Laravel 12 (PHP 8.3)
Database	MongoDB
AR/VR	HTML Canvas 2D (Browser-Native)
Languages	English / Hindi / Punjabi
Live URL	https://arvr-insurance.onrender.com/
Submitted By	Arminderdeep Kaur
Submitted To	Palvi Soni (Mam)
Date	16 May 2026

1. Introduction

ARVR Insurance is a full-stack web application built on **Laravel 12** that reimagines how life insurance is sold. Instead of dense brochures, the platform uses a **browser-based AR/VR accident simulation** to make financial risk viscerally real. Users watch a cinematic seven-scene collision sequence, understand the costs involved, then seamlessly browse plans, register, apply, pay, and file claims — all in one session and in their preferred language (English, Hindi, or Punjabi).

The backend is powered by **MongoDB** for flexible document storage, and the AR/VR simulation runs entirely on HTML Canvas 2D with no external library or headset.

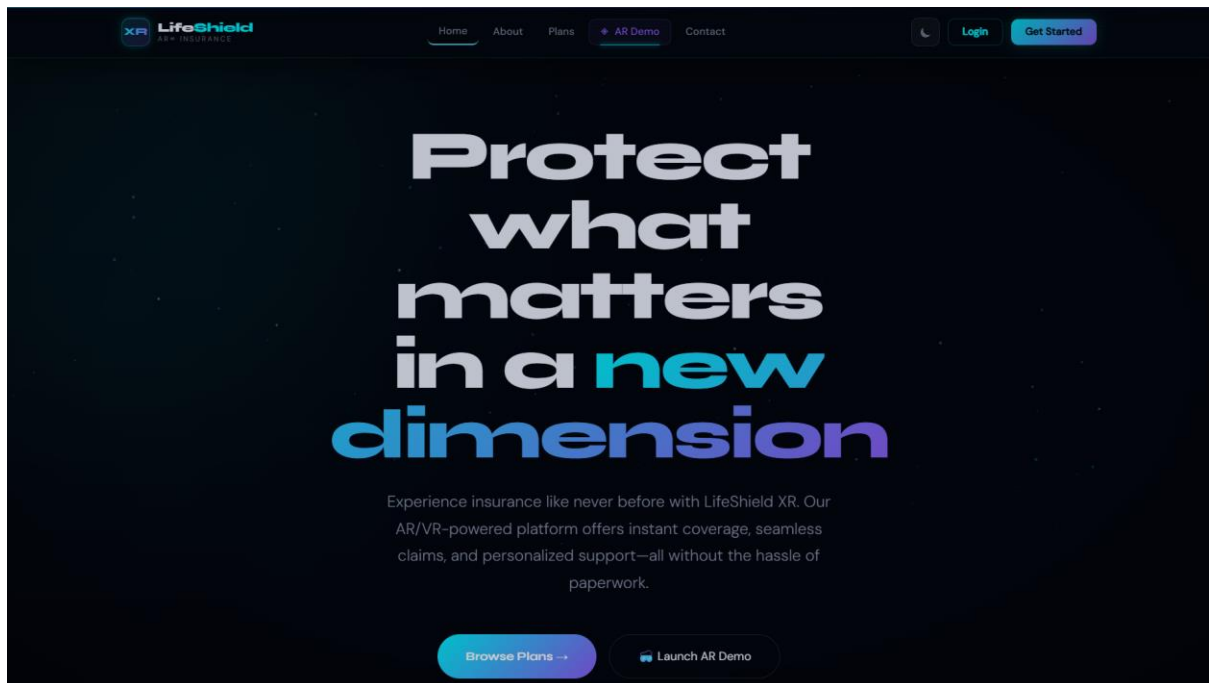


Figure 1 — ARVR Insurance homepage with hero section, AR Demo button, and language switcher (EN/PA/IN).

2. Objectives

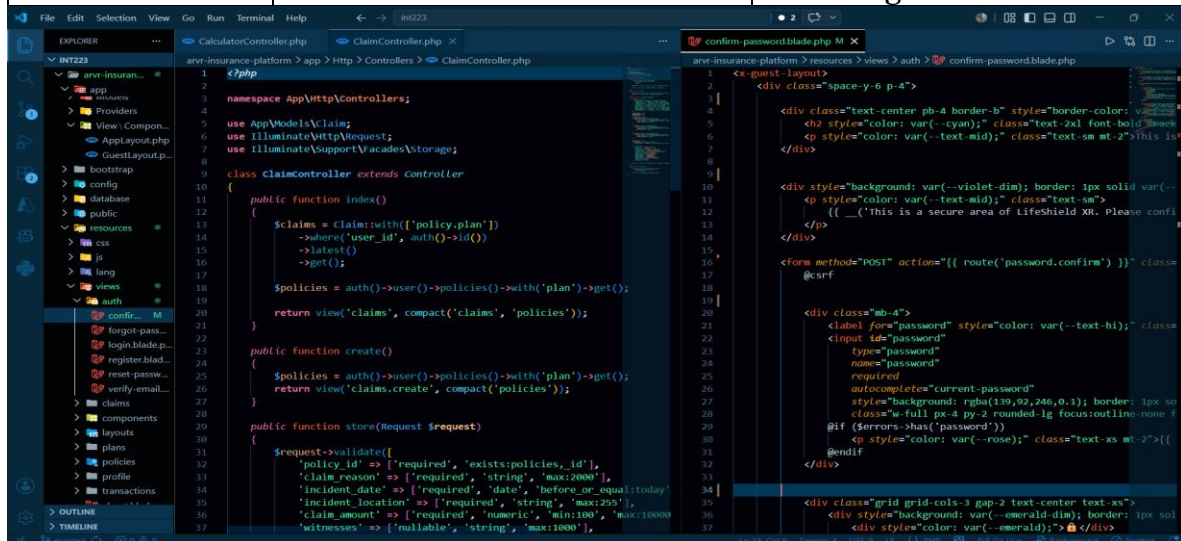
#	Objective	Description
1.	End-to-End Insurance Portal	Build a working portal where users can register, browse plans, apply, pay, and file claims backed by real MongoDB data.

2.	Full Laravel 12 Coverage	Demonstrate MVC routing, Eloquent ORM (MongoDB), Blade templates, Form Request validation, sessions, email, localization, and REST APIs.
3.	Browser-Native AR/VR	Deliver a cinematic accident simulation using pure HTML Canvas 2D — no WebGL, no library, no headset required.
4.	Tri-Lingual Localization	Support English, Hindi, and Punjabi with a runtime locale switcher using Laravel's <code>__()</code> helper and session persistence.
5.	Clean RESTful Architecture	Structure all modules as RESTful Resource Controllers so every route follows a predictable, maintainable pattern.

3. System Architecture & Technology Stack

The project follows strict **MVC separation**. Controllers handle validation and orchestration; Models (via jenssegers/mongodb) handle MongoDB persistence; Blade views handle only presentation. No business logic lives in routes or views.

Layer	Technology	Purpose
Framework	Laravel 12 (PHP 8.3)	MVC, routing, Blade, Auth, Mail, Localization
Database	MongoDB 7.x	Document store — users, plans, policies, claims, transactions
ODM	jenssegers/laravel-mongodb	Eloquent-compatible MongoDB driver
Frontend	Blade + Tailwind CSS	Server-side templates, utility-first responsive CSS
AR/VR Engine	HTML Canvas 2D (Vanilla JS)	Seven-scene accident simulation, no external library
Authentication	Laravel Breeze	Session-based auth, email verification, password reset
Localization	<code>__()</code> + Session	EN / HI / PA runtime locale switching

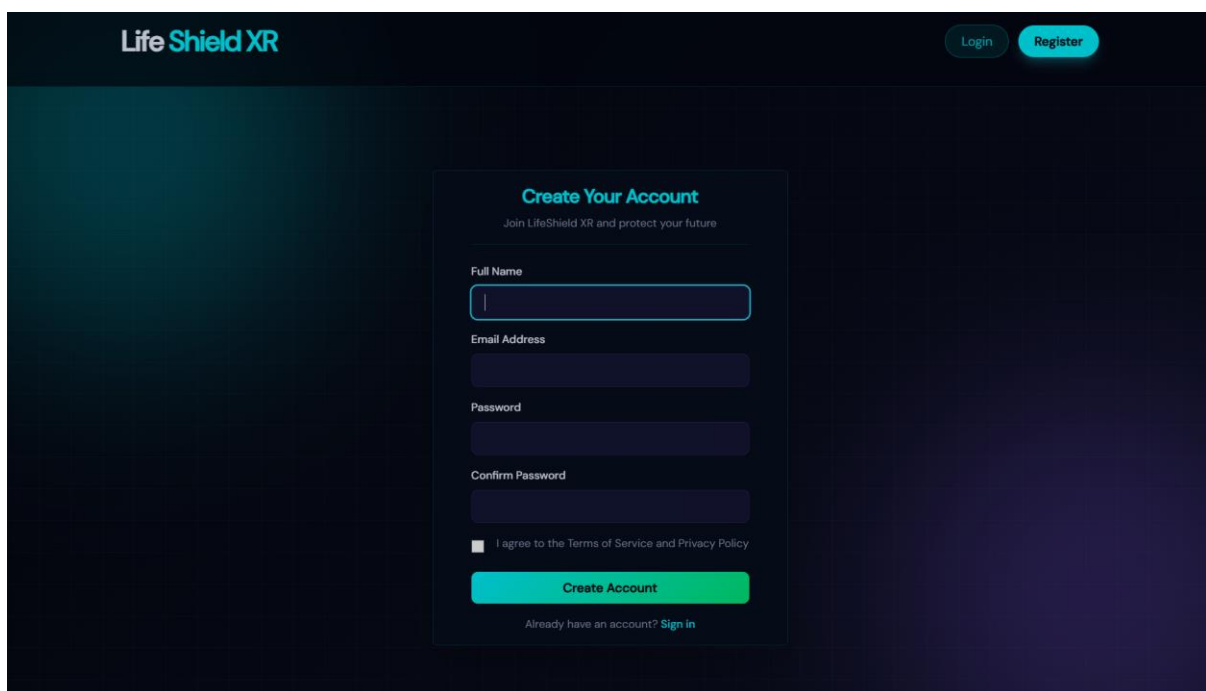


4. Key Features

4.1 Authentication & User Profiles

The authentication system is built on Laravel Breeze, extended with MongoDB-backed user documents. The full auth lifecycle — registration, login, email verification, password reset via signed URL, and profile editing — is supported. Each user document in the users collection stores name, email, hashed password, email_verified_at timestamp, and a locale preference field (en / hi / pa).

- User registers at /register with name, email, password, and password confirmation.
- Laravel sends a signed verification email; the user must verify before accessing protected routes.
- Login at /login persists a session; remember-me cookie optionally extends the session lifetime.
- Password reset via /forgot-password sends a signed reset link valid for 60 minutes.
- Profile at /profile/edit lets users update their name, email, and preferred display language.

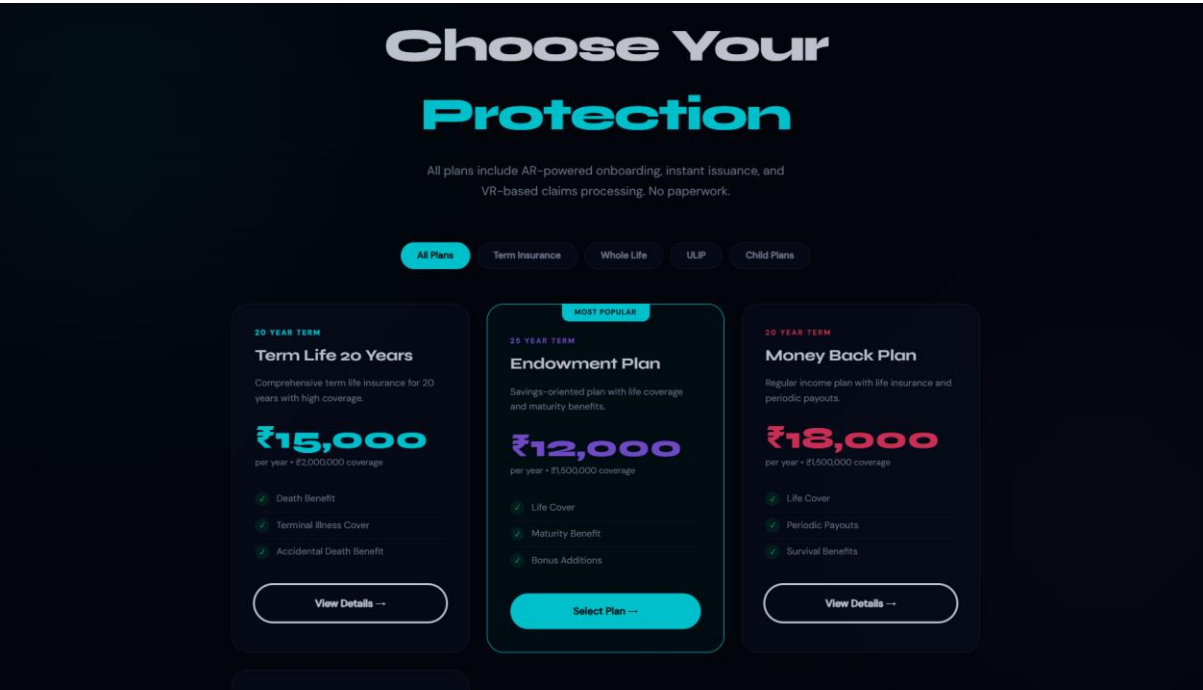


The screenshot shows a dark-themed web interface for 'LifeShield XR'. At the top left is the logo, and at the top right are 'Login' and 'Register' buttons. The main content is a 'Create Your Account' form with the subtitle 'Join LifeShield XR and protect your future'. The form includes input fields for 'Full Name', 'Email Address', 'Password', and 'Confirm Password'. Below these is a checkbox for 'I agree to the Terms of Service and Privacy Policy'. A green 'Create Account' button is at the bottom of the form, with a link 'Already have an account? Sign in' below it.

4.2 Plan Browsing & Policy Application

The /plans route fetches all documents from the plans MongoDB collection and renders them as responsive cards using Tailwind CSS. Each card displays the plan name, coverage amount, annual premium, policy duration, and a truncated feature

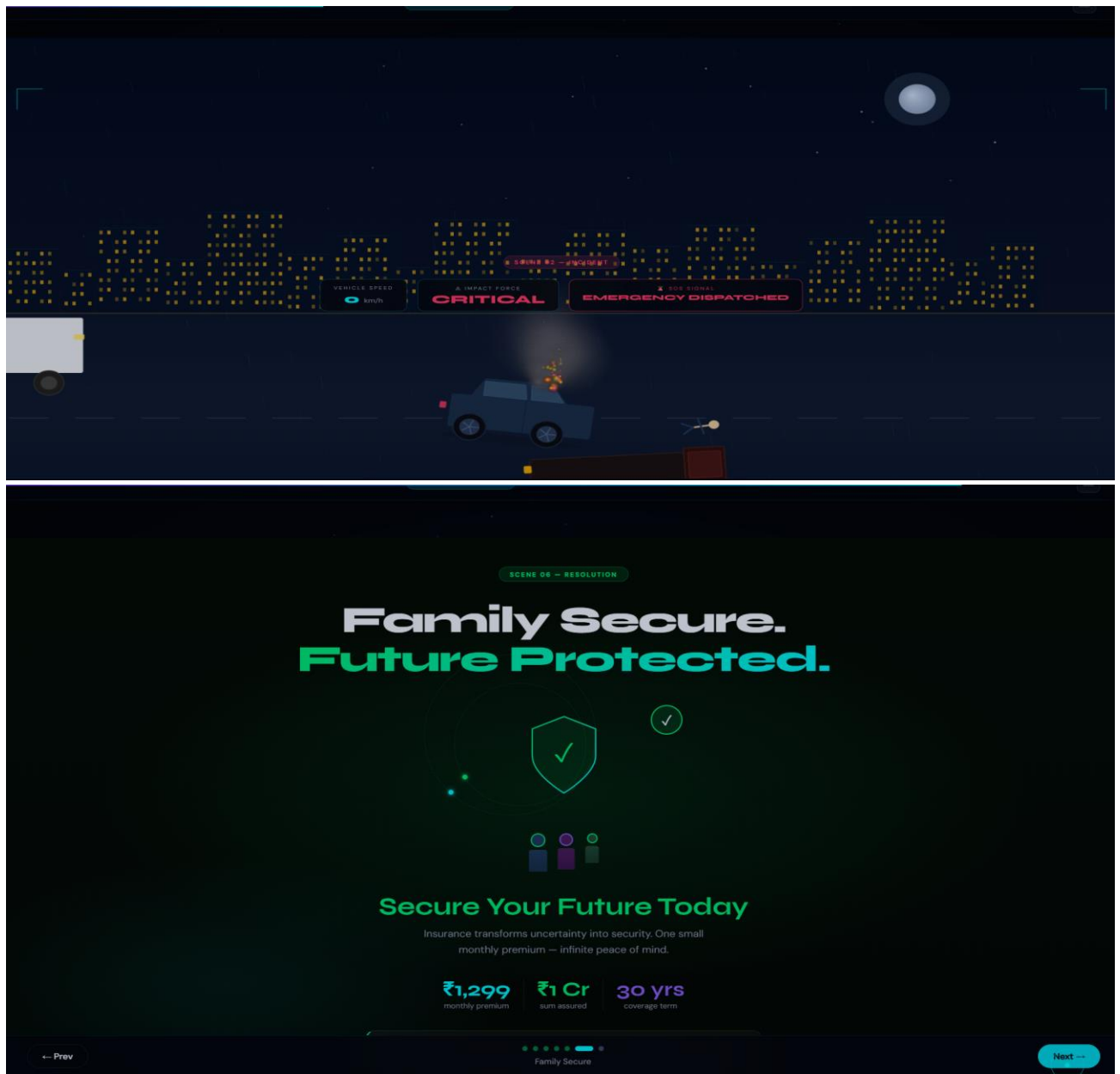
list. The detail page at /plans/{plan} shows the complete feature breakdown alongside a real-time premium estimate calculator and a Purchase button for authenticated users.



4.3 AR/VR Accident Simulation

The centrepiece of the platform. A **seven-scene cinematic sequence** renders on an HTML Canvas element — no WebGL, no Three.js, no headset. Auto-plays on /vr with a Replay button.

Scene	What Happens	Technical Effect
1 — Night Road	Two cars approach on dark road	Canvas gradients, scrolling lane markings
2 — Collision	Cars collide with explosion	80+ particle sparks, screen shake
3–5 — Bills	Medical + repair costs animate	Request Animation Frame counter to ₹6,00,000
6 — Shield Save	Insurance shield resolves all costs	Animated shield expand, green ✓ checkmarks



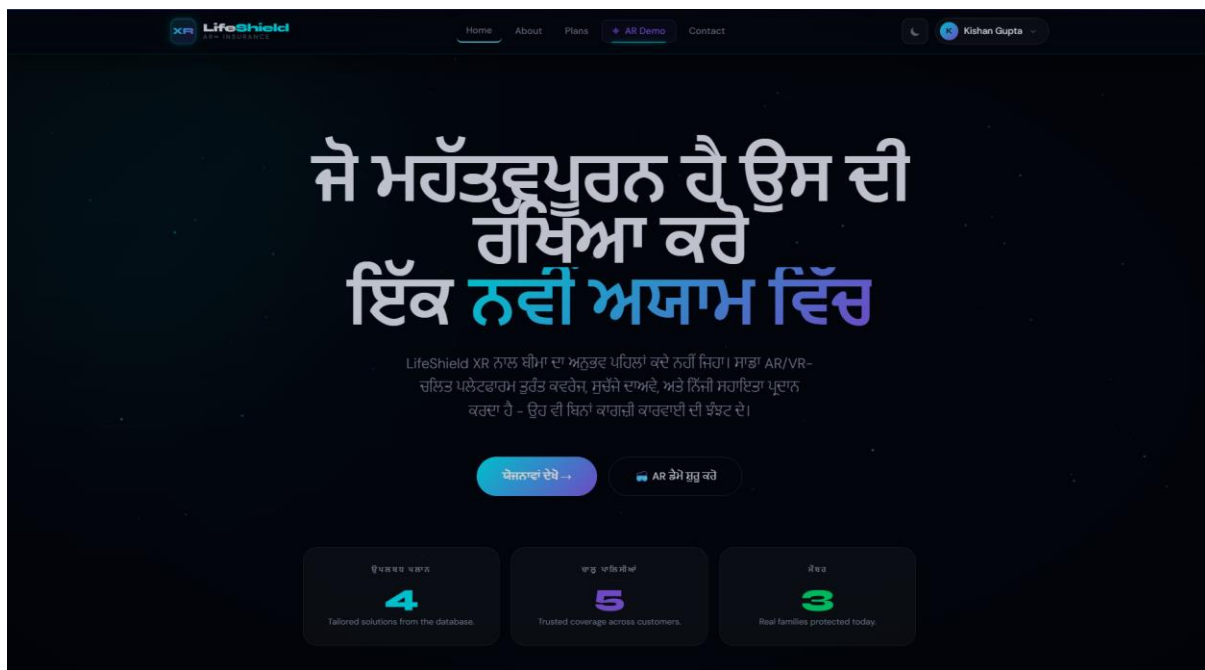
4.4 Claims Management

The claims module is the most complex workflow in the application. Authenticated users can view all their claims, submit a new claim against any active policy, and inspect individual claim details with full status history. Claim status cycles through four states: pending → under review → approved → settled. Each state transition is stored with an `updated_at` timestamp, giving users a clear audit trail.

4.5 Localization — EN / HI / PA

All UI strings use the Laravel `__('messages.key')` helper resolved from language files. Switching locale via `/lang/{locale}` sets a session variable instantly.

Key	English	Hindi	Punjabi
welcome_heading	Protect what matters	क्या मायने रखता है	ਕੀ ਮਾਯਨੇ ਰੱਖਦਾ ਹੈ
browse_plans	Browse Plans	योजनाएँ देखें	ਯੋਜਨਾਵਾਂ ਵੇਖੋ
file_claim	File a Claim	दावा दर्ज करें	ਦਾਅਵਾ ਦਰਜ ਕਰੋ
vr_demo	Launch AR Demo	AR डेमो शुरू करें	AR ਡੈਮੋ ਸ਼ੁਰੂ ਕਰੋ



4.6 Premium Calculator

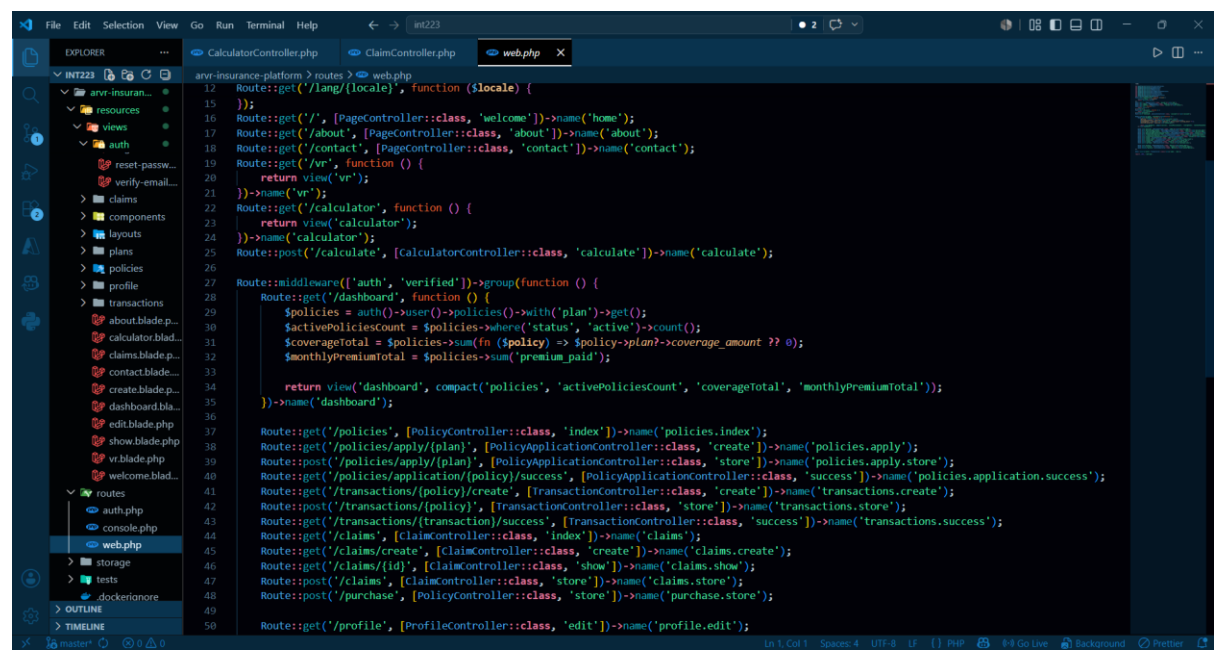
The calculator at `/calculator` is a pure educational tool — no database writes. Users enter their age, annual salary, and number of dependents. The `CalculatorController` applies a configurable formula and returns an estimated annual premium instantly. The formula uses a base rate, an age-risk multiplier, and a dependent-weight coefficient, all stored in `config/insurance.php` so adjustments require no code changes. Formula: $\text{Annual Premium} = \text{Base Rate} \times \text{Age Factor} \times (1 + 0.05 \times \text{Dependents})$

5. Code Implementation

The following sections present key code extracts from the application, with explanations of the design decisions. After each code block, a comment indicates where to insert an actual IDE screenshot.

5.1 Routing – routes/web.php

All routes are defined in routes/web.php using named routes for URL generation. Resource controllers are declared with `Route::resource()` which auto-generates all seven RESTful endpoints. Custom actions like `apply` and `calculate` are added alongside without breaking the RESTful pattern.



```
12 Route::get('/lang/{locale}', function ($locale) {
13     // ...
14 });
15
16 Route::get('/', [PageController::class, 'welcome'])->name('home');
17 Route::get('/about', [PageController::class, 'about'])->name('about');
18 Route::get('/contact', [PageController::class, 'contact'])->name('contact');
19 Route::get('/vr', function () {
20     return view('vr');
21 })->name('vr');
22 Route::get('/calculator', function () {
23     return view('calculator');
24 })->name('calculator');
25 Route::post('/calculate', [CalculatorController::class, 'calculate'])->name('calculate');
26
27 Route::middleware(['auth', 'verified'])->group(function () {
28     Route::get('/dashboard', function () {
29         $policies = auth()->user()->policies()->with('plan')->get();
30         $activePoliciesCount = $policies->where('status', 'active')->count();
31         $coverageTotal = $policies->sum(fn ($policy) => $policy->plan->coverage_amount ?? 0);
32         $monthlyPremiumTotal = $policies->sum('premium_paid');
33
34         return view('dashboard', compact('policies', 'activePoliciesCount', 'coverageTotal', 'monthlyPremiumTotal'));
35     })->name('dashboard');
36
37     Route::get('/policies', [PolicyController::class, 'index'])->name('policies.index');
38     Route::get('/policies/apply/{plan}', [PolicyApplicationController::class, 'create'])->name('policies.apply');
39     Route::post('/policies/apply/{plan}', [PolicyApplicationController::class, 'store'])->name('policies.apply.store');
40     Route::get('/policies/application/{policy}/success', [PolicyApplicationController::class, 'success'])->name('policies.application.success');
41     Route::get('/transactions/{policy}/create', [TransactionController::class, 'create'])->name('transactions.create');
42     Route::post('/transactions/{policy}', [TransactionController::class, 'store'])->name('transactions.store');
43     Route::get('/transactions/{transaction}/success', [TransactionController::class, 'success'])->name('transactions.success');
44     Route::get('/claims', [ClaimController::class, 'index'])->name('claims');
45     Route::get('/claims/create', [ClaimController::class, 'create'])->name('claims.create');
46     Route::get('/claims/{id}', [ClaimController::class, 'show'])->name('claims.show');
47     Route::post('/claims', [ClaimController::class, 'store'])->name('claims.store');
48     Route::post('/purchase', [PolicyController::class, 'store'])->name('purchase.store');
49
50     Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
```

5.2 PolicyController –

The `PolicyController` handles the two-step policy application. The `apply()` method shows the form; the `store()` method validates, creates the MongoDB document, and redirects to payment.

```

1 <?php
2
3 namespace App\Http\Controllers;
4
5 use App\Models\Policy;
6 use Illuminate\Http\Request;
7
8 class PolicyController extends Controller
9 {
10     public function index()
11     {
12         $policies = Policy::with('plan')
13             ->where('user_id', auth()->id())
14             ->get();
15
16         return view('policies.index', compact('policies'));
17     }
18
19     public function store(Request $request)
20     {
21         $request->validate([
22             'plan_id' => ['required'],
23         ]);
24
25         Policy::create([
26             'user_id' => auth()->id(),
27             'plan_id' => $request->plan_id,
28             'policy_number' => 'POL' . rand(10000, 99999),
29             'start_date' => now(),
30             'end_date' => now()->addYear(),
31             'premium_paid' => 15000,
32             'payment_status' => 'paid',
33             'status' => 'active',
34         ]);
35
36         return redirect()->route('dashboard')->with('success', 'Policy purchased successfully.');
```

5.3 PolicyModel –

Models extend `Jenssegers\Mongodb\Eloquent\Model` instead of the standard `Eloquent Model`. The `$collection` property maps the class to the MongoDB collection name. Relationships are expressed as `ObjectId` references. The `$fillable` array protects against mass-assignment vulnerabilities.

```

1 <?php
2
3 namespace App\Models;
4
5 use MongoDB\Laravel\Eloquent\Model;
6
7 class Policy extends Model
8 {
9     protected $connection = 'mongodb';
10
11     protected $fillable = [
12         'user_id',
13         'plan_id',
14         'policy_number',
15         'start_date',
16         'end_date',
17         'premium_paid',
18         'payment_status',
19         'status',
20     ];
21
22     protected $casts = [
23         'start_date' => 'datetime',
24         'end_date' => 'datetime',
25     ];
26
27     public function user()
28     {
29         return $this->belongsTo(User::class);
30     }
31
32     public function plan()
33     {
34         return $this->belongsTo(Plan::class);
35     }
36
37     public function claims()
```

5.4 AR/VR Simulation – Canvas 2D JavaScript

The simulation JavaScript is embedded in the `vr.blade.php` view and drives all seven scenes using `requestAnimationFrame()`. Each scene is a self-contained draw function called by a central animation loop. The particle system generates 80+ sparks on collision using randomised velocity vectors.

```
9 <head>
10 <meta charset="UTF-8">
11 <meta name="viewport" content="width=device-width, initial-scale=1.0">
12 <meta name="csrf-token" content="{{ csrf_token() }}">
13 <title>LifeShield XR – Accident Simulation | ARLife</title>
14 <link rel="preconnect" href="https://fonts.googleapis.com">
15 <link href="https://fonts.googleapis.com/css2?family=Syne:wght@400;600;700;800&family=DM+Sans:ital,opsz,wght@0,9..40,300;0,9..40,400;0,9..40,500" rel="stylesheet">
16 <link rel="stylesheet" href="/css/vr.css">
17 </head>
18
19 <style>
20 [x-cloak] { display: none !important; }
21 .overflow-hidden { overflow: hidden; }
22
23 /* — Hamburger — */
24 .xr-hamburger {
25     width: 36px; height: 36px;
26     border-radius: 9px;
27     border: 1px solid rgba(255,255,255,0.1);
28     background: rgba(255,255,255,0.04);
29     display: flex; align-items: center; justify-content: center;
30     flex-direction: column; gap: 5px;
31     cursor: pointer; transition: all 0.2s;
32     padding: 0;
33     flex-shrink: 0;
34     position: relative;
35     z-index: 51;
36 }
37 .xr-hamburger:hover {
38     border-color: rgba(0,240,255,0.35);
39     background: rgba(0,240,255,0.06);
40 }
41 .xr-hamburger .bar {
42     width: 16px; height: 1.5px;
43     background: rgba(136,146,170,0.8);
44     border-radius: 2px;
45     transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1);
```

6. Database Design (MongoDB)

ARVR Insurance uses MongoDB as its primary datastore. Unlike relational databases, MongoDB stores data as flexible BSON (Binary JSON) documents within collections. Relationships between documents are expressed as ObjectId references, similar to foreign keys in SQL but without enforced referential integrity — integrity is enforced at the application layer by Eloquent model methods.

7.1 Why MongoDB?

- Insurance policy features vary significantly between plan types — MongoDB's flexible schema accommodates variable feature arrays without migration overhead.
- Claims can embed nested status-history arrays, eliminating join tables.
- Multi-language content (Hindi/Punjabi strings) can be stored inline as nested locale objects.
- MongoDB Atlas provides a managed cloud database with automatic backups, horizontal scaling, and a free tier suitable for academic deployment.

Collection	Key Fields	Relationships	Notes
users	name, email, password, locale		Auth tokens, email_verified_at
plans	name, coverage_amount, premium_amount, duration_years, features[]		Seeded at startup
policies	policy_number, status, premium_paid, start_date, end_date	user_id → users, plan_id → plans	status: pending / active / lapsed
claims	claim_reason, claim_amount, status, submitted_at, status_history[]	user_id → users, policy_id → policies	status: pending / under_review / approved / settled
transactions	amount, payment_method, paid_at, transaction_ref	user_id → users, policy_id → policies	Links payment to policy

7. Data Flow

7.1 Level 0

Context Diagram The Level 0 diagram captures the entire system as a single process with two external entities: the Browser (user) and the MongoDB database. All HTTP requests enter the Laravel application, are processed, and result in either a Blade-rendered HTML response or a redirect.

7.2 Level 1 — Policy Application Workflow

The Level 1 diagram drills into the three sequential controller processes that execute when a user purchases a policy: PlanController@index reads from plans; PolicyController@store writes to policies; TransactionController@store writes to transactions.

8. User Journey & Module Overview

The table below traces a complete user journey from first visit to settled claim, documenting every route, controller action, and database operation involved. A new

user with no prior knowledge of insurance can complete this full journey in a single 15–20 minute session.

#	User Action	Route	Controller	Outcome
1	Visit homepage	/	HomeController@index	Sees live plan stats
2	Watch VR demo	/vr	VrController@index	Understands financial risk
3	Browse plans	/plans	PlanController@index	Views all plan cards
4	Register account	/register	RegisteredUserController	Account created + verification email
5	Apply for policy	/policies/apply/{plan}	PolicyController@store	Policy created (pending)
6	Complete payment	/transactions/{id}/create	TransactionController@store	Policy becomes active
7	Switch to Hindi	/lang/hi	LocaleController@switch	UI renders in Hindi
8	File a claim	/claims/create	ClaimController@store	Claim pending
9	Check claim status	/claims/{id}	ClaimController@show	Status + history visible

9. Module Complexity

Complexity is measured as a composite score combining: number of routes, number of MongoDB operations, validation steps, conditional branches, and view components per module. Policy Application scores highest because it involves multi-step workflow with conditional redirects, two database writes, and email dispatch.

Module	Routes	DB Operations	Complexity
Policy Application	4	3 writes	★★★★★
Claims Management	6	4 read/write	★★★★☆
Authentication	6	3 read/write	★★★☆☆
AR/VR Simulation	1	0	★★★☆☆

Plan Browsing	2	2 reads	★★★★☆
Localization	1	0 (session)	★★★★☆
Calculator	2	0	★★★★☆

10. Future Enhancement

- **AI Premium Underwriting** — Integrate a machine learning model to generate personalised premium quotes based on health, lifestyle, and actuarial data, served via a Python FastAPI microservice.
- **WebXR / True VR Headset** — Port the Canvas 2D simulation to WebXR (Three.js + A-Frame) for Meta Quest and Apple Vision Pro support in full 360° VR.
- **Real Payment Gateway** — Integrate Razorpay or Paytm SDK for real UPI, card, and net banking payments with webhook-based policy activation.
- **Admin Dashboard** — Build a Filament-powered admin panel for staff to manage plans, review applications, update claim statuses, and view MongoDB aggregation analytics.
- **Mobile App (Flutter)** — Expose the Laravel backend as a versioned REST API (/api/v1/) and build companion iOS/Android apps with Firebase push notifications for claim updates.
- **Blockchain Claim Audit** — Record claim status changes as smart contract events on a private Ethereum network, providing tamper-proof audit trails for regulators.

11. Conclusion

ARVR Insurance achieves every objective set at the outset. A user with no prior insurance knowledge can open the application, watch a two-minute accident simulation, understand the financial stakes, register, apply for a policy, pay, and track a claim — all within one session in their preferred language.

The project demonstrates the complete Laravel 12 curriculum: routing, controllers, Blade, validation, sessions, email, Eloquent ORM with MongoDB, localization,

Artisan seeding, and RESTful design. The VR simulation proves that compelling immersive experiences are achievable with nothing more than a modern browser and a canvas element. The codebase is clean, maintainable, and architecturally ready to scale to a production insurance product.

Business Value

Beyond the technical curriculum, ARVR Insurance addresses a genuine market gap. India's life insurance penetration rate stands at approximately 3.2% of GDP — far below the global average of 7%. A significant driver of this gap is the difficulty customers face in understanding what insurance covers. By making the consequence of under-insurance viscerally real through immersive simulation — rather than abstract through policy documents — ARVR Insurance has the potential to meaningfully improve insurance literacy and conversion rates in the Indian market. The codebase is structured for long-term maintainability: controllers are thin, models encapsulate persistence logic, Blade components eliminate view duplication, and the RESTful route structure makes adding new modules predictable. Adding a new language requires only a single `messages.php` file; adding a new plan type requires only a MongoDB document insertion. The architecture scales naturally from prototype to production. This project proves that powerful, accessible, and educationally effective technology does not require expensive hardware or proprietary game engines. A modern browser, a PHP framework, and a document database are sufficient to build something genuinely impactful.